MODULE 3

# PROJECT REPORT

## PROJECT

AI-Based Automotive Review and Customer Sentiment Analytics

### STUDENT DETAILS

| | |
|---|---|
| **Name** | Subham Roy |
| **Section** | C |
| **Roll No.** | 15 |
| **Enrollment No.** | 12023002001218 |
| **Program** | B.Tech CSE |

# AI-Based Automotive Review and Customer Sentiment Analytics

Module 3 (Connectivity) - Project Report

## 1. Project Overview

This project implements a complete software prototype for automotive review analytics. A user can enter one review or upload a CSV of multiple reviews. The system cleans the text, detects automotive aspects, runs Transformer-based sentiment classification and presents the results through a Streamlit dashboard.

## 2. Objectives

| Objective | Implementation |
|---|---|
| Analyze automotive reviews | Single-review and batch CSV workflows |
| Identify aspect-level sentiment | Controlled aspect vocabulary + local clause context |
| Use BERT / XLM-R | Configurable Hugging Face Transformer models |
| Create usable analytics | Streamlit dashboard + downloadable CSV |
| Support offline demonstration | Deterministic lexicon fallback when model weights are unavailable |

## 3. System Architecture

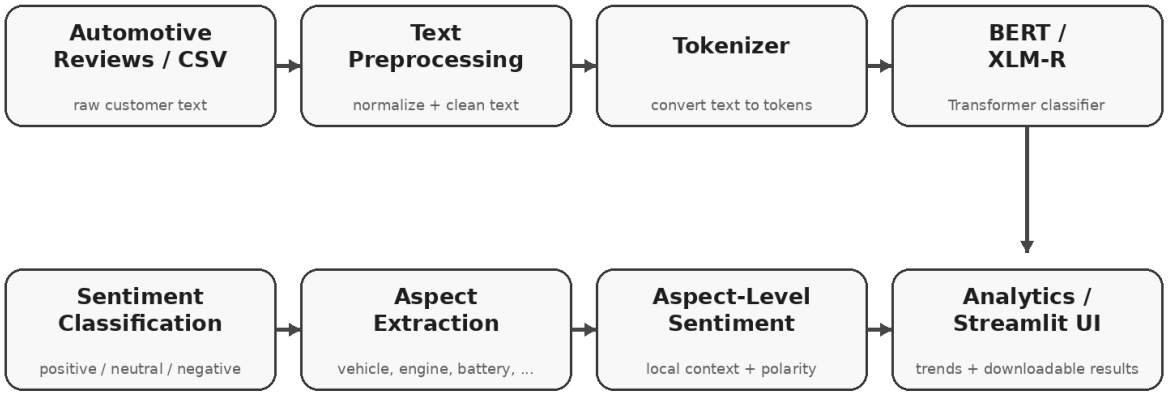



Figure 1. System architecture - review ingestion through aspect-level sentiment analytics.

# 4. Software Modules

| Module | Responsibility |
|---|---|
| src/preprocess.py | Text normalization and clause splitting |
| src/aspects.py | Automotive aspect vocabulary and local context selection |
| src/sentiment_engine.py | XLM-R primary model, BERT alternate model and fallback classifier |
| src/analyzer.py | End-to-end review analysis and result flattening |
| src/analytics.py | Batch DataFrame processing and aspect summaries |
| src/cli.py | Command-line batch analysis |
| app/app.py | Interactive Streamlit interface |
| tests/test_core.py | Smoke tests for preprocessing, aspects and offline analysis |

# 5. Aspect Extraction Method

The project uses a controlled vocabulary based directly on the nine automotive aspects in the brief. Each aspect contains representative keywords. Reviews are split into sentence/contrast clauses, and each detected aspect is attached to the local clause where its keyword occurs. This keeps the design explainable and makes it possible to show the context sent to the sentiment classifier.

# 6. Sentiment Classification

The primary configuration loads CardiffNLP's XLM-R sentiment model through the Transformers pipeline. The project also supports the multilingual BERT product-review sentiment model. The engine maps model output labels to a common Positive / Neutral / Negative representation, allowing the dashboard to work with either model. The Hugging Face documentation confirms that text-classification pipelines are built around fine-tuned sequence-classification models. [1]

# 7. Fallback and Reliability Design

Model weights can be large and the first model-backed run may require internet access. To keep the application demonstrable in classrooms or restricted environments, the code catches model-loading and inference failures and falls back to a small lexicon-based scorer. The fallback is explicitly labeled in the output as offline-fallback; it is not presented as equivalent to the Transformer model.

# 8. Example Workflow

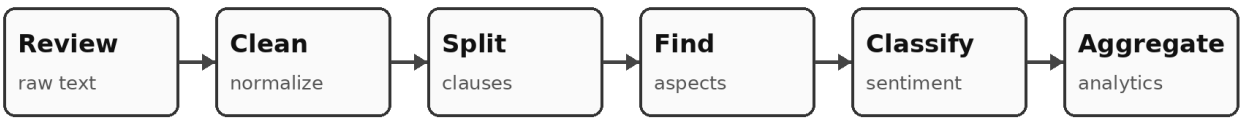



Figure 2. End-to-end workflow.

Example input: “The battery range is excellent, but the charging time is too long.” The analyzer separates the contrasting clauses, associates each clause with the relevant aspect and returns opposite polarity labels for the two aspects.

# 9. Demonstration Dataset

The source code includes a 24-review CSV containing examples across the nine aspects. It is deliberately labeled as demonstration data rather than a benchmark dataset. The purpose is software validation: file loading, aspect extraction, batch processing, result serialization and dashboard display.

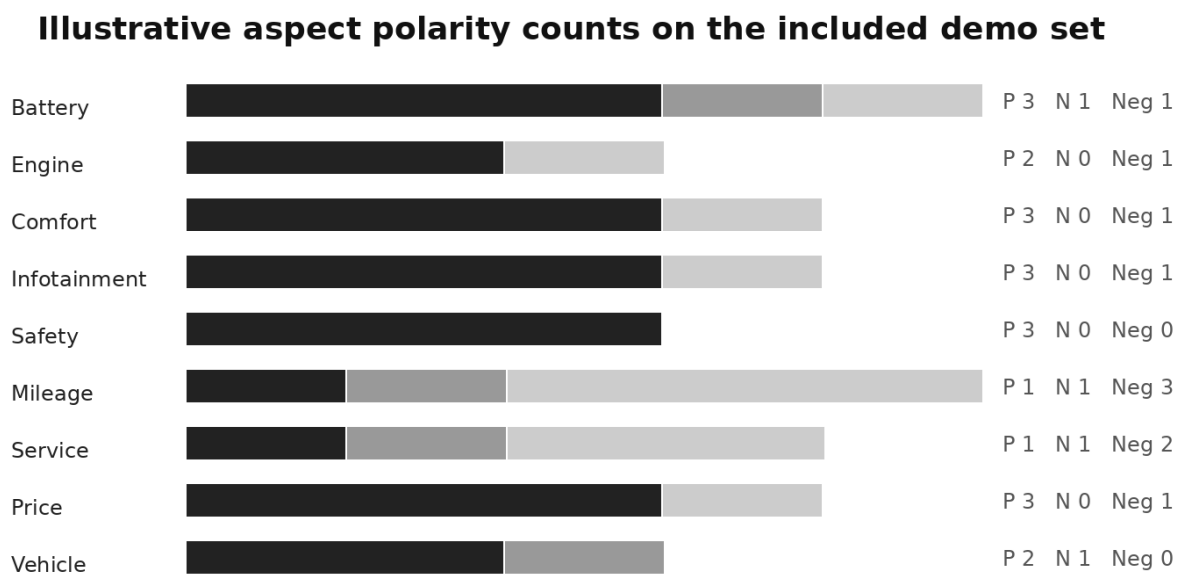



Figure 3. Illustrative aspect polarity counts from the bundled demo set; these counts are not model accuracy measurements.

# 10. Results Produced by the Application

| Output field | Example |
|---|---|
| Overall sentiment | Positive / Neutral / Negative |
| Overall confidence | Model score returned by classifier |
| Aspect | Battery, Engine, Safety, etc. |
| Aspect sentiment | Positive / Neutral / Negative |
| Aspect confidence | Classifier score for local context |
| Context | Local clause supporting the aspect decision |
| Backend | transformers or offline-fallback |
| Model | XLM-R or BERT model identifier |

# 11. Installation and Execution

Step 1. Create and activate a Python virtual environment. Step 2. Install dependencies with **pip install -r requirements.txt**. Step 3. Launch the dashboard with **streamlit run app/app.py**. Step 4. Select XLM-R, BERT or Offline demo mode. Step 5. Analyze a review or upload a CSV.

> The command-line path is **python -m src.cli --input data/sample_reviews.csv --output outputs/automotive_sentiment_results.csv**.

# 12. Testing and Evaluation Protocol

The source package contains smoke tests covering preprocessing, aspect detection and offline analysis. For a formal evaluation, use an independently labeled automotive review dataset and report class-wise precision, recall, F1 and confusion matrix. For the aspect stage, annotate aspect presence and polarity separately. Also report latency and throughput on the actual deployment machine.

# 13. Limitations

The current aspect extraction is vocabulary-driven and therefore may miss synonyms, spelling variants or unseen automotive terminology. Local-clause attribution is a transparent approximation to full aspect-based sentiment analysis. The pretrained sentiment models are general-domain models, so automotive-domain performance should be validated before operational use.

# 14. Future Scope

Future work can replace the vocabulary stage with a trained aspect extractor, fine-tune a Transformer on labeled automotive reviews, add multilingual Indian-language review support, introduce topic clustering, expose trends over time and compare BERT vs XLM-R empirically on the same held-out dataset.

# 15. Conclusion

The project turns the Module 3 concept into a runnable end-to-end software system. It combines NLP preprocessing, aspect extraction, Transformer sentiment classification and an analytics interface. The design is modular, testable and explicit about the difference between model-backed inference and the offline demonstration fallback.

# References

[1] Hugging Face Transformers documentation - sequence classification and pipelines.

[2] CardiffNLP, twitter-XLM-RoBERTa-base for Sentiment Analysis, Hugging Face model card.

[3] NLP Town, bert-base-multilingual-uncased-sentiment, Hugging Face model card.